Independent University, Bangladesh

CSE313 · Every True / False statement, most important first

Tier 1 is asked in every past paper. Write T or F and a one-line reason for each; the reason earns the mark.

Question paper

A. Tier 1 · Which grammar works with which parser (asked in every past paper)

1. All TDP can use Left Recursive Grammar (LRG). T / F
2. All TDP can use Non-deterministic Grammar (NDG). T / F
3. Left Recursive Grammar (LRG) can not be used in LL(1). T / F
4. Non-deterministic Grammar (NDG) can not be used in LL(1). T / F
5. An LL(1) grammar can be left recursive. T / F
6. A grammar with a FIRST/FIRST conflict can be parsed by an LL(1) parser. T / F
7. If two rules fall in one cell of the table, the grammar is not LL(1). T / F
8. Brute Force Parser is a BUP. T / F
9. Recursive Descent Parser (RDP) is as powerful as LL(1). T / F
10. Recursive Descent Parser (RDP) uses OS stack. T / F
11. Recursive Descent Parser (RDP) can use Non-deterministic Grammar (NDG). T / F
12. A predictive parser needs backtracking. T / F
13. From LRG grammar complete First set can't be found. T / F
14. From LRG grammar complete Follow set can't be found. T / F

B. Tier 2 · Conversions and ambiguity

15. Left factoring removes left recursion. T / F
16. Left factoring changes the language of the grammar. T / F
17. Removing left recursion changes the language of the grammar. T / F
18. Converting an ambiguous NDG to a DG (left factoring) makes it unambiguous. T / F
19. Removing left recursion from an ambiguous grammar makes it unambiguous. T / F
20. An ambiguous grammar can be LL(1). T / F
21. Every LL(1) grammar is unambiguous. T / F
22. In an unambiguous expression grammar, + is defined at a higher level (closer to the start symbol) than *. T / F
23. A left recursive rule (E → E + T) makes the operator left associative. T / F
24. The rule for ^ (power) should be left recursive. T / F

C. Tier 3 · FIRST, FOLLOW and the LL(1) table

25. The FIRST set of a non-terminal can contain ε. T / F
26. The FOLLOW set of a non-terminal can contain ε. T / F
27. $ is always in FOLLOW of the start symbol. T / F
28. FIRST of a terminal is the terminal itself. T / F
29. For A → ε, the rule is placed in M[A, b] for every b in FOLLOW(A). T / F
30. ε can be a column in the LL(1) parsing table. T / F
31. An LL(1) parsing table has one row for each terminal. T / F
32. In LL(1), the first L means the input is scanned Left to right. T / F

D. Tier 4 · Parser classes, phases and lexical analysis

33. A Bottom-Up Parser builds the parse tree from the root down to the leaves. T / F
34. Bottom-up parsing uses the reverse of the rightmost derivation. T / F
35. The lexical analyzer removes comments and white space. T / F
36. The lexical analyzer builds the parse tree. T / F
37. Syntax analysis comes after lexical analysis. T / F
38. Semantic analysis belongs to the analysis (front end) part of the compiler. T / F
39. An interpreter produces a target program in machine code. T / F
40. Regular expressions can check balanced parentheses. T / F

Answers (exam layout, with comments)

Tier 1 · Which grammar works with which parser (asked in every past paper)

WRITE THIS

#	Statement	Answer	Reason (one line)	Seen in
1	All TDP can use Left Recursive Grammar (LRG).	**False**	Left recursion makes a top-down parser loop forever (A ⇒ Aα ⇒ Aαα …).	Aut-23 3(a)
2	All TDP can use Non-deterministic Grammar (NDG).	**False**	Predictive parsers (LL(1)) need a deterministic grammar. Only backtracking parsers cope with NDG.	Aut-23 3(b)
3	Left Recursive Grammar (LRG) can not be used in LL(1).	**True**	LL(1) is top-down; left recursion loops and puts two rules in one cell.	Sum-23 4(a)
4	Non-deterministic Grammar (NDG) can not be used in LL(1).	**True**	Alternatives with the same prefix fall in the same cell (FIRST/FIRST conflict).	Sum-23 4(b)
5	An LL(1) grammar can be left recursive.	**False**	Left recursion must be removed first.	sample 3(a)
6	A grammar with a FIRST/FIRST conflict can be parsed by an LL(1) parser.	**False**	Two rules in one cell, so the grammar is not LL(1).	sample 3(c)
7	If two rules fall in one cell of the table, the grammar is not LL(1).	**True**	The parser cannot choose with one look-ahead.	drill
8	Brute Force Parser is a BUP.	**False**	Brute force = top-down parsing with backtracking.	Aut-23 3(e)
9	Recursive Descent Parser (RDP) is as powerful as LL(1).	**False**	RDP with backtracking tries every alternative, so it accepts grammars LL(1) cannot (more general, but slower).	Sum-23 4(c)
10	Recursive Descent Parser (RDP) uses OS stack.	**True**	One procedure per non-terminal, called recursively, so it uses the program (OS) call stack.	Sum-23 4(d)
11	Recursive Descent Parser (RDP) can use Non-deterministic Grammar (NDG).	**True**	With backtracking it tries one alternative, and on failure goes back and tries the next.	Sum-23 4(e)
12	A predictive parser needs backtracking.	**False**	A predictive parser never backtracks. It uses the look-ahead and the table.	drill
13	From LRG grammar complete First set can't be found.	**False**	For A → Aα | β, FIRST(A) = FIRST(β) (plus FIRST(α) if β can be ε). Textbook view; write the reason.	Aut-23 3(c)
14	From LRG grammar complete Follow set can't be found.	**False**	FOLLOW comes from where a symbol appears on right sides. Left recursion does not stop this.	Aut-23 3(d)

Tier 2 · Conversions and ambiguity

WRITE THIS

#	Statement	Answer	Reason (one line)	Seen in
15	Left factoring removes left recursion.	**False**	Left factoring removes common prefixes (NDG → DG). Left recursion needs A → βA', A' → αA' | ε.	set B / drill
16	Left factoring changes the language of the grammar.	**False**	It only rewrites the rules; the set of strings stays the same.	sample 3(e)
17	Removing left recursion changes the language of the grammar.	**False**	The new grammar makes exactly the same strings. Only the shape of the trees changes.	drill
18	Converting an ambiguous NDG to a DG (left factoring) makes it unambiguous.	**False**	Left factoring only delays the choice; every parse tree maps one-to-one, so ambiguity stays.	Sum-23 Q3
19	Removing left recursion from an ambiguous grammar makes it unambiguous.	**False**	E → E + E | id becomes E → id E', E' → + E E' | ε, and id + id + id still has two LMDs.	set C Q3
20	An ambiguous grammar can be LL(1).	**False**	Two trees for a string means two rules compete for some cell.	set B 3(b)
21	Every LL(1) grammar is unambiguous.	**True**	Each cell has at most one rule, so each string has only one derivation.	set A 3(a)
22	In an unambiguous expression grammar, + is defined at a higher level (closer to the start symbol) than *.	**True**	Lower precedence = upper level of the grammar.	drill
23	A left recursive rule (E → E + T) makes the operator left associative.	**True**	The tree grows to the left, so the left operation is done first.	lesson 6
24	The rule for ^ (power) should be left recursive.	**False**	^ is right associative, so its rule is right recursive: F → G ^ F.	drill

Tier 3 · FIRST, FOLLOW and the LL(1) table

WRITE THIS

#	Statement	Answer	Reason (one line)	Seen in
25	The FIRST set of a non-terminal can contain ε.	**True**	If A ⇒* ε then ε ∈ FIRST(A).	sample 3(b)
26	The FOLLOW set of a non-terminal can contain ε.	**False**	FOLLOW holds terminals and $ only; ε never goes in FOLLOW.	set A 3(b)
27	$ is always in FOLLOW of the start symbol.	**True**	Rule 1 of FOLLOW.	drill
28	FIRST of a terminal is the terminal itself.	**True**	FIRST rule 1.	drill
29	For A → ε, the rule is placed in M[A, b] for every b in FOLLOW(A).	**True**	ε rules go under the FOLLOW terminals of the left side.	set B 3(c)
30	ε can be a column in the LL(1) parsing table.	**False**	Columns are terminals and $. ε rules are placed using FOLLOW.	drill
31	An LL(1) parsing table has one row for each terminal.	**False**	One row per non-terminal; one column per terminal plus $.	set A 3(c)
32	In LL(1), the first L means the input is scanned Left to right.	**True**	L = left-to-right scan, L = leftmost derivation, 1 = one look-ahead.	set B 3(e)

Tier 4 · Parser classes, phases and lexical analysis

WRITE THIS

#	Statement	Answer	Reason (one line)	Seen in
33	A Bottom-Up Parser builds the parse tree from the root down to the leaves.	**False**	BUP goes from the leaves (input) up to the root. TDP goes root down.	set A 3(e)
34	Bottom-up parsing uses the reverse of the rightmost derivation.	**True**	Slides: bottom-up uses the reverse of rightmost derivation.	drill
35	The lexical analyzer removes comments and white space.	**True**	The scanner throws them away while making tokens.	sample 3(d)
36	The lexical analyzer builds the parse tree.	**False**	The lexer makes tokens. The syntax analyzer (parser) builds the parse tree.	drill / set B 3(d)
37	Syntax analysis comes after lexical analysis.	**True**	The parser reads the tokens that the lexical analyzer makes.	set A 3(d)
38	Semantic analysis belongs to the analysis (front end) part of the compiler.	**True**	Lexical, syntax and semantic analysis form the analysis part.	drill
39	An interpreter produces a target program in machine code.	**False**	An interpreter does not produce a target program; it executes directly.	drill
40	Regular expressions can check balanced parentheses.	**False**	Finite automata cannot remember nesting depth. A CFG is needed.	drill